

massive

Lecture 12 Scaling to data 3 Dec 2025

EPFL Clément Pit-Claudel / Robert West

Announcements

Friday's lab session:

- Exercises on scaling to massive data ([Exercise 10](#))
- Project office hour (same [sign-up protocol](#) as before)

2

Today's lecture: Prof. Clément Pit-Claudel



Systems and Formalisms Lab



3

Feedback

Give us feedback on this lecture here:

<https://go.epfl.ch/ada2025-lec12-feedback>

- What did you (not) like about this lecture?
- What was (not) well explained?
- On what would you like more (fewer) details?
- ...

4

Warm up

```
count = 0
for n in range(0, 10**9):
    if n.bit_count() == 5:
        count += 1
print(count)
```

How long does this program take?

5

Warm up

One year of Moodle logs:

```
14000 students
x 3-20 sessions per day
x 4-7 days per week
x 14 weeks per term
x 2 terms per year
x 3-5 pages per session
x 10-80 requests per page (!)
x 200-500 bytes per log entry
180.188.241.108 - - [24/May/2015:13:55:18 +0000] "GET /moodle/ HTTP/1.1" 200 176 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/42.0.2311.135 Safari/537.36"
≈ 5500 GB
≈ 10 hours to read from a hard drive??
```

⇒ distribute, parallelize, stream

So far in this class...

- We made one big assumption:
 - All data fits on a single machine
 - Even more, all data fits into memory on a single machine (Pandas)
- Realistic assumption for **prototyping**, but frequently not for production code

The takeaway before we even start

- Good programmers speak their computer's language.
- A data scientist's computer is the cloud.

⇒ You need to be able to program the cloud.

The big-data problem

Data is growing faster than computation speed

- + Growing data sources (e.g. Web, mobile, sensors, ...)
- + Cheap hard-disk storage

- Slow read speeds ⇒ Must distribute
- Stalling CPU speeds ⇒ Must parallelize
- RAM bottlenecks ⇒ Must stream



Examples

- All StackOverflow posts: [100 GB](#)
- WSL (yay!) rockslide measurements: [2 TB](#) / day
- Worldwide stock-market data : [2.5 TB](#) / day
- Github Arctic Code Vault: [21 TB](#)
- Facebook daily logs, 2016: [60 TB](#) / day
- Entire BrainMaps project: [140 TB](#)
- Uber's analytics: [PBs](#) / day
- CERN's filtered experimental data: [1 PB](#) / day
- Amadeus (air traffic data): [43 PB](#)
- Entire Wayback Machine archive: [99 PB](#)
- CERN raw experimental data: [1 PB](#) / second

Typical hard drive:
\$12 / TB (2024)
150 MB/s reads
⇒ 2h / TB

But how much data should you get?

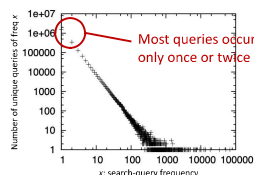
Of course, "it depends", but for many applications the answer is:

As much as you can get

Big data about people (text, Web, social media) tends to follow heavy-tailed distributions (e.g., power laws)

Example: Web search

59% of all Web search queries are unique
17% of all queries are made only twice
8% are made three times



The big-data problem

A single machine can no longer store, let alone process, all the data

The only solution is to **distribute** over a large cluster of machines

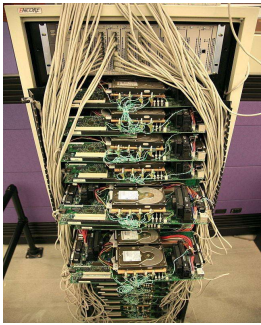
Hardware for big data

Budget (a.k.a. commodity) hardware
Not “gold-plated” (a.k.a. custom)

Many low-end servers
Easy to add capacity
Cheaper per CPU and per disk

Increased complexity in software:

- Fault tolerance
- Virtualization (e.g., distributed file systems)



Google Corkboard server: Steve Jurvetson/Flickr

Problems with cheap hardware

Failures, e.g. (Google numbers)

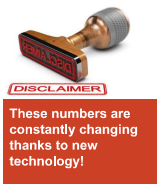
- 1–5% hard drives/year
- 0.2% DIMMs (dual in-line memory modules)/year

Commodity network (1–10 Gb/s) speeds vs. RAM

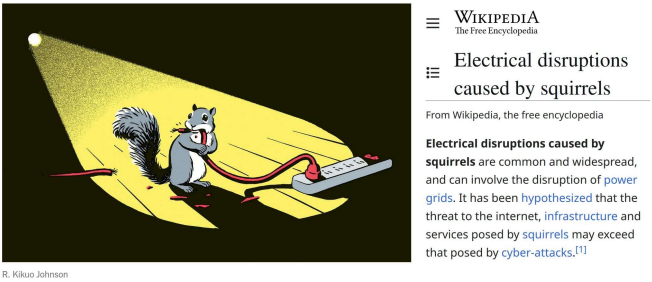
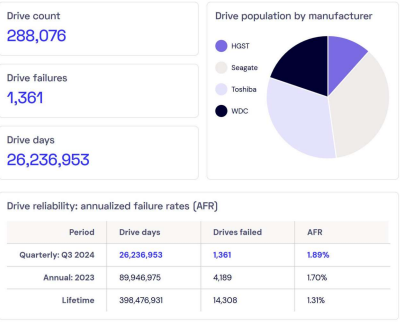
- Much more latency (100–100,000x)
- Lower throughput (100–1,000x)

Uneven performance

- Inconsistent hardware (e.g., old + new)
- Variable network latency
- External loads

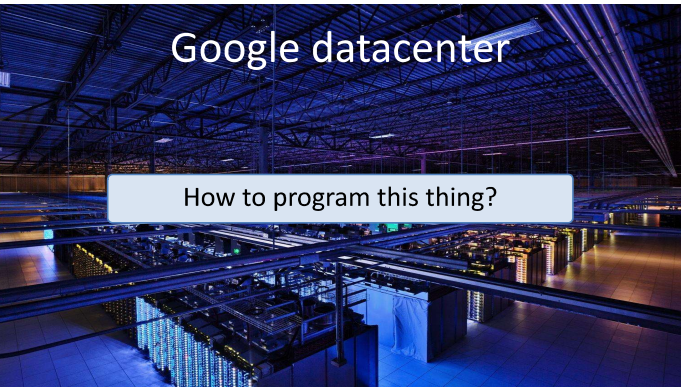


Drive Stats snapshot as of Q3 2024



R. Kikuo Johnson

Want to know more about datacenter failures? Watch “[Frying Squirrels and Unspun Gyros](#)”, or just look up “squirrel outage” online



Google datacenter

How to program this thing?

What’s hard about cluster computing?

1. How to split work across machines?
2. How to deal with failures?

4 models of distributed computation

- Distributed aggregation (intuition only)
- MapReduce (historical)
- Spark RDDs
- Spark high-level APIs

Models of distributed computation

Distributed aggregation (intuition)

19

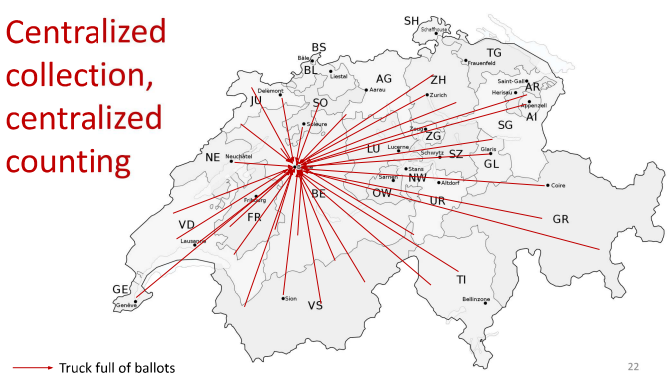
20

A real world example: Counting votes



21

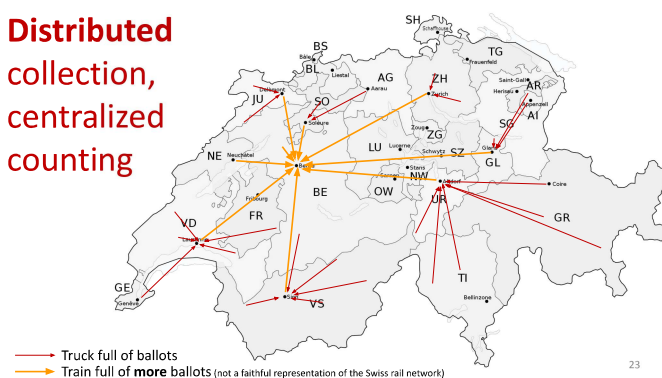
Centralized
collection,
centralized
counting



→ Truck full of ballots

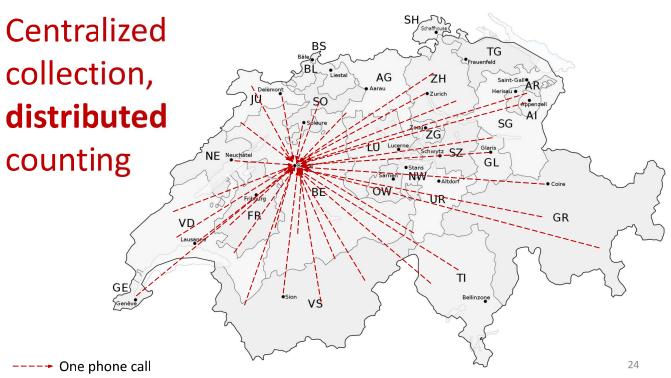
22

Distributed
collection,
centralized
counting



23

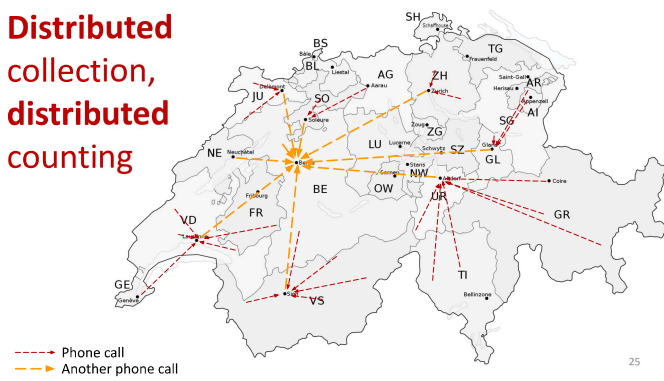
Centralized
collection,
distributed
counting



-----➔ One phone call

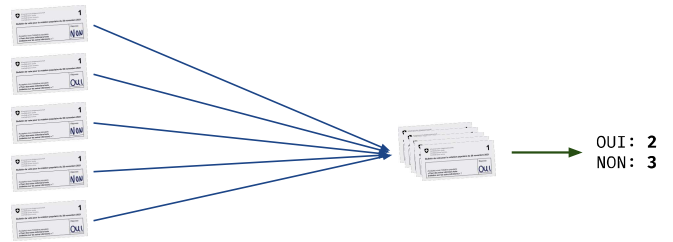
24

Distributed collection, distributed counting



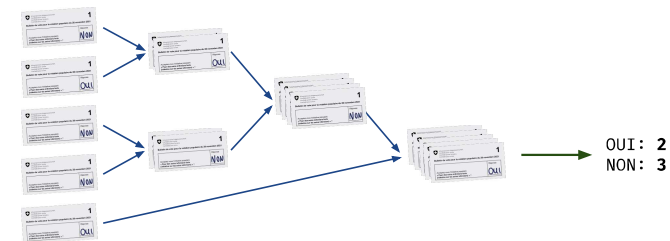
25

All centralized



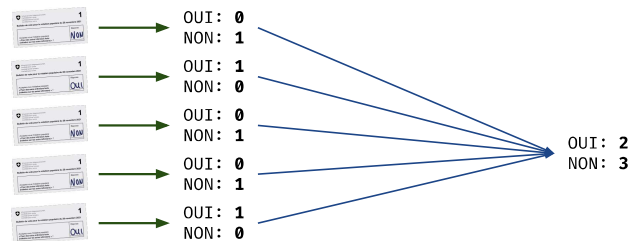
26

Distributed collection



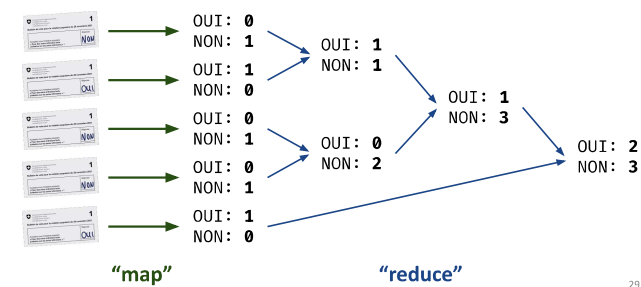
27

Distributed counting



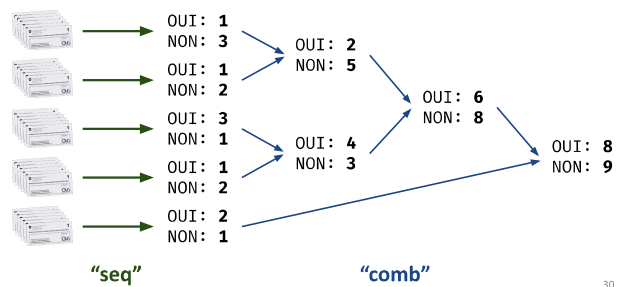
28

All distributed



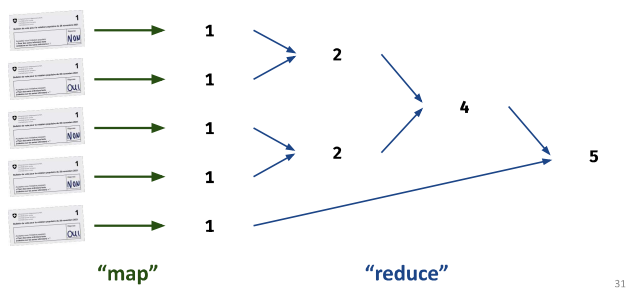
29

All distributed + local proc.



30

Exercise: distributed count as map+reduce?



Models of distributed computation

MapReduce / Hadoop
(no, this isn't *just* map+reduce)

32

How do you count the number of occurrences of each word in a document?

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

→

I: 3
am: 3
Sam: 3
do: 1
you: 1
like: 1
...

A hashtable (a.k.a. dict)!

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

}

A hashtable!

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

{I: 1}

A hashtable!

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

{I: 1,
am: 1}

A hashtable!

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

{I: 1,
am: 1,
Sam: 1}

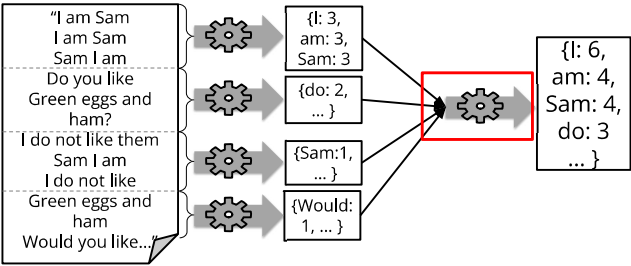
A hashtable!

"I am Sam
I am Sam
Sam I am
Do you like
Green eggs and
ham?"

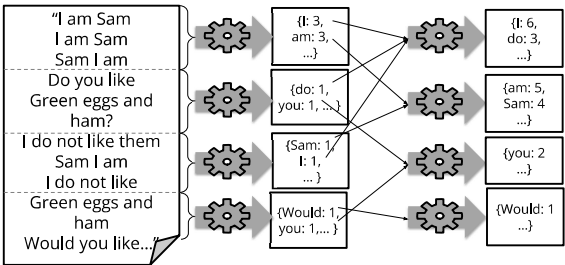
{I: 2,
am: 1,
Sam: 1}

What if the document is
REALLY big?

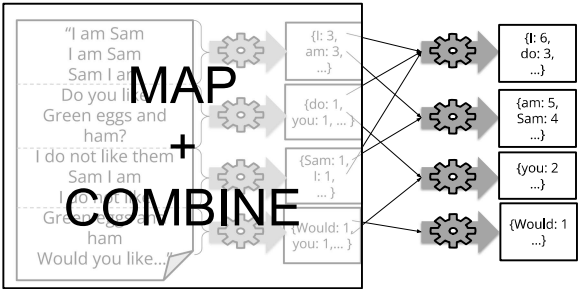
What if the document is
really big?



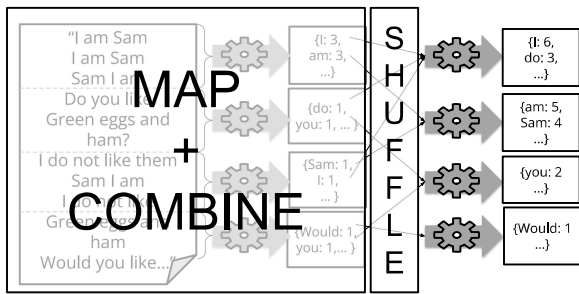
“Divide and Conquer”



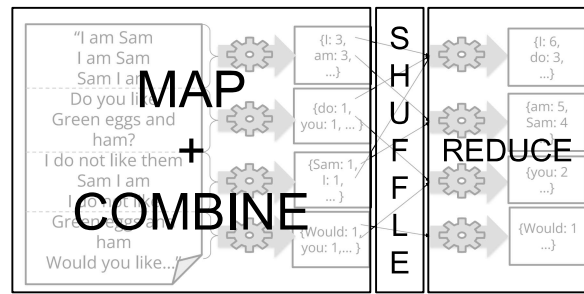
“Divide and Conquer”



“Divide and Conquer”



“Divide and Conquer”



Recall: What’s hard about cluster computing?

- How to divide work across machines?
 - Moving data may be very expensive
 - Must consider network, data locality
- How to deal with failures?
 - 1 server fails every 3 years $\Rightarrow$ 10K servers see ~10 faults/day
 - Even worse: stragglers (node not failed, but slow)

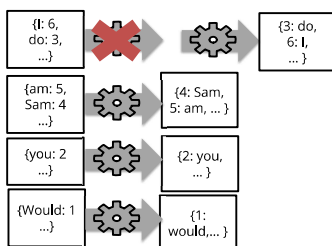
Solution: MapReduce

- Smart systems engineers have done all the work for you
 - Task scheduling
 - Virtualization of file system
 - Fault tolerance (incl. data replication)
 - Job monitoring
 - etc.
- “All” you need to do: implement Mapper and Reducer classes



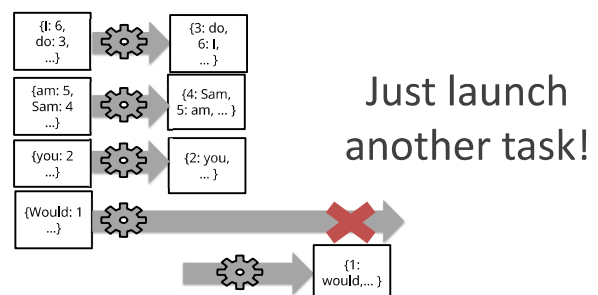
46

How to deal with failures?



Just launch another task!

How to deal with slow tasks?





Solution: MapReduce

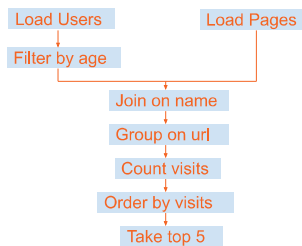
- Smart systems engineers have done all the work for you
 - Task scheduling
 - Virtualization of file system
 - Fault tolerance (incl. data replication)
 - Job monitoring
 - etc.
- "All"** you need to do: implement Mapper and Reducer classes
 - Need to break more complex jobs into sequence of MapReduce jobs



50

Example task

Suppose you have user info in one file, website logs in another, and you need to find the top 5 pages most visited by users aged 18–25



Example from <http://wiki.apache.org/pig-data/attachments/PigTalksPapers/attachments/ApacheConEurope09.ppt>

51

In MapReduce



52

The takeaway before we even start

- Good programmers speak their computer's language.
- A data scientist's computer is the cloud.

⇒ You need to be able to program the cloud.

Enter: Spark



- A high-level fun(ctional) programming API


```

print("I am a regular Python program, using pyspark")

users = (sc.textFile('users.tsv') # user <TAB> age
        .map(lambda line: line.split('\t'))
        .filter(lambda user_age: 18 <= int(user_age[1]) <= 25))

pages = (sc.textFile('pageviews/*.tsv') # user <TAB> url
        .map(lambda line: line.split('\t')))

counts = (users.join(pages)
          .map(lambda user_age_url: (user_age_url[1][1], 1))
          .reduceByKey(lambda x, y: x + y)
          .takeOrdered(5))
      
```

54

Enter: Spark



- A high-level fun(ctional) programming API

```

print("I am a regular Python program, using pyspark")

users = (sc.textFile('users.tsv') # user <TAB> age
        .map(lambda line: line.split('\t'))
        .filter(lambda user_age: 18 <= int(user_age[1]) <= 25))

pages = (sc.textFile('pageviews/*.tsv') # user <TAB> url
        .map(lambda line: line.split('\t')))

counts = (users.join(pages)
          .map(lambda user_age_url: (user_age_url[1][1], 1))
          .reduceByKey(lambda x, y: x + y)
          .takeOrdered(5))
  
```

55

Break

Ada

🔍

📄 ⭐ ✎

🔗 Ovo je glavno značenje pojma **Ada**. Za druga značenja pogledajte **Ada** (razdvajba).

Ada je hrvatski naziv za riječni otok i čest toponim u sjevernoj Hrvatskoj. Riječni otoci nastaju tako što u sporijem dijelu riječnog toka **taloženjem nanosa** mogu nastati i riječni **sprudovi**. Takvi sprudovi katkad mogu izbijati i iznad površine.

Najveći riječni otok je **Marajó** površine od oko 40 tisuća km², koji se nalazi na delti rijeke **Amazona**.^[1] Najmanji naseljeni riječni otok je **Umananda** na rijeci **Brahmaputri**.

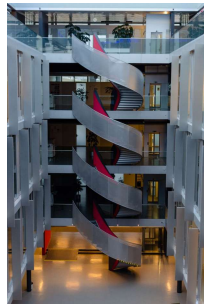
Najpoznatiji primjer ađe u Hrvatskoj jest **Šarengradska ada** na **Dunavu** koja ima površinu od 6.7 km² te je tako i najveća ada u Hrvatskoj.^[1]

Sam naziv **ada** potječe iz turskoga jezika.



Ada na rijeci Ohlju.

56



- Implemented in  **Scala** (go EPFL!)
- Additional APIs in
 - Python
 - Java
 - R
- Many abstractions
 - Core: RRDs
 - Derived: dataframes, SQL, ML toolkit

57

Models of distributed computation

RDDs: concepts (this one has a proper map+reduce!)

58

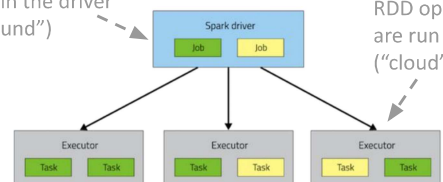
RDD: resilient distributed dataset

- To programmer: looks like one single collection (each element represents a “row” of a dataset)
- Under the hood: oh boy...
 - RDDs “live in the cloud”: split over several machines, replicated, etc.
 - Can be processed in parallel
 - Can be transformed to a single, real collection (if small...)
 - Typically read from a distributed file system (HDFS)
 - Can be written to a distributed file system

59

architecture

Your Python script runs in the driver (“ground”)



RDD operations are run in executors (“cloud”)

60

RDD operations



- “Transformations”
 - Input: RDD; output: another RDD
 - Everything remains “in the cloud”
 - Example: for every entry in the input RDD, count chars
 - RDD: {‘I’, ‘am’, ‘you’} → RDD: {1, 2, 3}
- “Actions”
 - Input: RDD; output: a value that is returned to the driver
 - Result is transferred “from cloud to ground”
 - Examples: take a sample of entries from RDD and print it on the driver’s shell; or store results to file (local or distributed)

61

Lazy execution

- **Transformations** (i.e., RDD→RDD operations) are not executed until it’s really necessary (a.k.a. “lazy execution”)
- Execution of transformations triggered by **actions**
- Why?
 - If you never look at the data, there’s no point in manipulating it...
 - Smarter query processing possible:
E.g., `rdd2 = rdd1.map(f1)`
`rdd3 = rdd2.filter(f2)`
Can be done in one go — no need to materialize `rdd2`

62

Models of distributed computation

RDDs in practice: transformations, actions, broadcasting, accumulators

63

RDD transformations [\(full list\)](#)

- **map**(*func*): Return a new distributed dataset formed by passing each element of the source through a function *func*
`{1,2,3}.map(lambda x: x*2) → {2,4,6}`
- **filter**(*func*): Return a new dataset formed by selecting those elements of the source on which *func* returns true
`{1,2,3}.filter(lambda x: x <= 2) → {1,2}`
- **flatMap**(*func*): Similar to map, but each input item can be mapped to 0 or more output items (so *func* should return a list rather than a single item)
`{1,2,3}.flatMap(lambda x: [x,x*10]) → {1,10,2,20,3,30}`

64

Exercises

1. Rewrite the following to be more efficient:

```
rdd.sort().first() ==  
rdd.map(f).map(g) ==
```

2. Implement map and filter using only flatMap

```
rdd.map(f) ==  
  
rdd.filter(f) ==
```

65

Exercises

1. Rewrite the following to be more efficient:

```
rdd.sort().first() == rdd.min()  
rdd.map(f).map(g) == rdd.map(lambda x: g(f(x)))
```

2. Implement map and filter using only flatMap

```
rdd.map(f) ==  
rdd.flatMap(lambda x: [f(x)])  
rdd.filter(f) ==  
rdd.flatMap(lambda x: [x] if f(x) else [])
```

66

RDD transformations [\[full list\]](#)

- **sample**(*withReplacement?*, *fraction*, *seed*): Sample a *fraction* of the data, with or without replacement, using a given random number generator *seed*
- **union**(*otherDataset*):
New dataset that contains the union of the source and the argument.
- **intersection**(*otherDataset*): ...
- **distinct**():
New dataset that contains distinct elements of the source.

67

RDD transformations [\[full list\]](#)

- **sample**(*withReplacement?*, *fraction*, *seed*): Sample a *fraction* of the data, with or without replacement, using a given random number generator *seed*

Why *relative fraction*,
and not *absolute number*?

68

RDD transformations [\[full list\]](#)

- **groupByKey**(): When called on a dataset of (K, V) pairs, returns a dataset of $(K, \text{Iterable}<V>)$ pairs.
`{(a, 7), (b, 2.1), (a, 1.3)}.groupByKey()`
 $\rightarrow \{(a, \{7, 1.3\}), (b, \{2.1\})\}$
- **reduceByKey**(*func*): When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function *func*, which must be of type $(V, V) \Rightarrow V$.
`{(a, 7), (b, 2.1), (a, 1.3)}.reduceByKey(Lambda (x, y): x+y)`
 $\rightarrow \{(a, 8.3), (b, 2.1)\}$
- **reduce**(*func*): ??

69

RDD transformations [\[full list\]](#)

- **sortByKey**(): When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs sorted by keys
- **join**(*otherDataset*): When called on datasets of type (K, V) pairs and (K, W) pairs, returns a dataset of $(K, (V, W))$ pairs with all pairs of elements for each key
`{(1, a), (2, b)}.join({(1, A), (1, X)})`
 $\rightarrow \{(1, (a, A)), (1, (a, X))\}$
- Analogous: **leftOuterJoin**, **rightOuterJoin**, **fullOuterJoin**
- (There are many more RDD transformations; cf. [tutorial](#))

70

RDD actions [\[full list\]](#)

- **collect**(): Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
- **count**(): Return the number of elements in the dataset.
- **reduce**(*func*): Compute a single value (**deterministic?**)
- **take**(*n*): Return an array with the “first” *n* elements of the dataset.
- **saveAsTextFile**(*path*): Save to local filesystem or HDFS.
- (There are several other RDD actions; cf. [tutorial](#))

71

Broadcast variables

- `my_set = set(...)`
`rdd2 = rdd1.filter(lambda x: x in my_set)`
^ This is a bad idea: `my_set` needs to be shipped with every task (one task per data partition, so if `rdd1` is spread over *N* partitions, the above will require copying the same object *N* times)
- `my_set = sc.broadcast(set(...))`
`rdd2 = rdd1.filter(lambda x: x in my_set.value)`
^ This way, `my_set` is copied to each executor only once and persists across all tasks (one per partition) on the same executor
- Broadcast variables are **read-only**

72

Accumulators

- `def f(x): return x*2`
`rdd2 = rdd1.map(f)`
^ How can we easily know how many rows there are in rdd1 (without running a costly reduce operation)?
- Side effects via accumulators!
`counter = sc.accumulator(0)`
`def f(x): counter.add(1); return x*2`
`rdd2 = rdd1.map(f)`
- Accumulators are **write-only** (“add-only”) for executors
- Only driver can read the value: `counter.value`

73

How does Spark run?

```
print("I am a regular Python program, using pyspark")

users = (sc.textFile('users.tsv') # user <TAB> age
        .map(lambda line: line.split('\t'))
        .filter(lambda user_age: 18 <= int(user_age[1]) <= 25))

views = (sc.textFile('pageviews/*.tsv') # user <TAB> url
        .map(lambda line: line.split('\t')))

counts = (users.join(views)
         .map(lambda user_age_url: (user_age_url[1][1], 1))
         .reduceByKey(lambda x, y: x + y)
         .takeOrdered(5))
```

74

RDD persistence

```
rdd2 = rdd1.map(f1)
list1 = rdd2.filter(f2).collect()
list2 = rdd2.filter(f3).collect()
```

} `rdd1.map(f1)` transformation is executed twice

```
rdd2 = rdd1.map(f1)
rdd2.persist()
list1 = rdd2.filter(f2).collect()
list2 = rdd2.filter(f3).collect()
```

} Result of `rdd1.map(f1)` transformation is cached and reused (can choose between memory and disk for caching)

75

Models of distributed computation

Dataframes and SQL (no need to think about map+reduce!)

76

Spark DataFrames

- Bridging the gap between your experience with Pandas and the need for distributed computing
 - RDD = collection of records
 - DataFrame = table with rows and typed columns
- Important to understand what RDDs are and what they offer, but today most of the tasks can be accomplished with DataFrames (**higher level of abstraction ⇒ less code**)
- [Getting Started with DataFrames | Databricks](#)

77

Example task

Suppose you have user info in one file, website logs in another, and you need to find the top 5 pages most visited by users aged 18–25

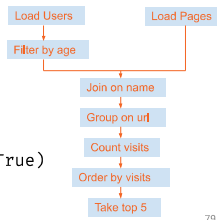


Example from <http://wiki.apache.org/pig-data/attachments/PigTalksPapers/attachments/ApacheConEurope09.ppt>

78

Spark DataFrames

```
users = spark.read.csv("users.csv", header=True)
views = spark.read.csv("pageviews/*.csv", header=True)
counts = (
    users[users.age.between(18, 25)]
    .drop("age")
    .join(views, on="name")
    .groupby("url").count()
    .sort_values("name", ascending=True)
    .head(5)
).toPandas()
```



79

Spark SQL [\[link\]](#)



```
counts = spark.sql("""
    SELECT url, COUNT(*)
    FROM users
    JOIN views ON users.name = views.name
    WHERE 18 ≤ age AND age ≤ 25
    GROUP BY url
""")

counts.toPandas()
```

80

Spark's Machine Learning Toolkit

MLlib: Algorithms [\[more details\]](#)

Classification

- Logistic regression, decision trees, random forests

Regression

- Linear (with L1 or L2 regularization)

Unsupervised:

- Alternating least squares
- K-means
- SVD
- Topic modeling (LDA)

Optimizers

- Optimization primitives (SGD, L-BGFS)

Example:

Logistic regression with MLlib

```
from pyspark.mllib.classification \
    import LogisticRegressionWithSGD

trainData = sc.textFile(" ... ").map( ... )
testData = sc.textFile(" ... ").map( ... )
model = LogisticRegressionWithSGD.train(trainData)
predictions = model.predict(testData)
```

Remarks

- This lecture is not enough to teach you Spark!
- To use it in practice, you'll need to delve into further online material. Read about RDDs, DataFrames, SQL, ...
- Also: Friday's lab session
- Important skill: assess whether you'd benefit from it
 - E.g., >1TB: yes, you'll need Spark
 - 20GB: it depends...



83

Feedback

Give us feedback on this lecture here:

<https://go.epfl.ch/ada2025-lec12-feedback>

- What did you (not) like about this lecture?
- What was (not) well explained?
- On what would you like more (fewer) details?
- ...

84